



VIT[®]
BHOPAL UNIVERSITY



A Project Report submitted as part of
Vityarthi E-Learning / Skill Development course

MATHEMATICAL PROBLEM SOLVER

Course Name:

CSE1021

Course Code: CSE1021

Submitted By:

Name : Lakshya Jatere
Registration No. : 26MIP10081
School : School of Computing Science
Engineering and Artificial Intelligence
(SCAI)

School

School of Computing Science
and Engineering and Artificial Intelligence (SCAI)
VIT Bhopal University

TABLE OF CONTENTS

S.No.	Topic/ Section Name	Page No.
1.	Introduction and Problem Overview	1
2.	Project objectives	2
3.	System Architecture and Folder Structure	3
4.	Module Breakdown and Description	4
5.	Operational Workflow and Flowchart	5
6.	Testing, Validation and Unit Tests	6
7.	Execution Outputs and Screenshots	7
8.	Conclusion and Future Scope	8

INTRODUCTION AND OBJECTIVES

1.1 Project Overview

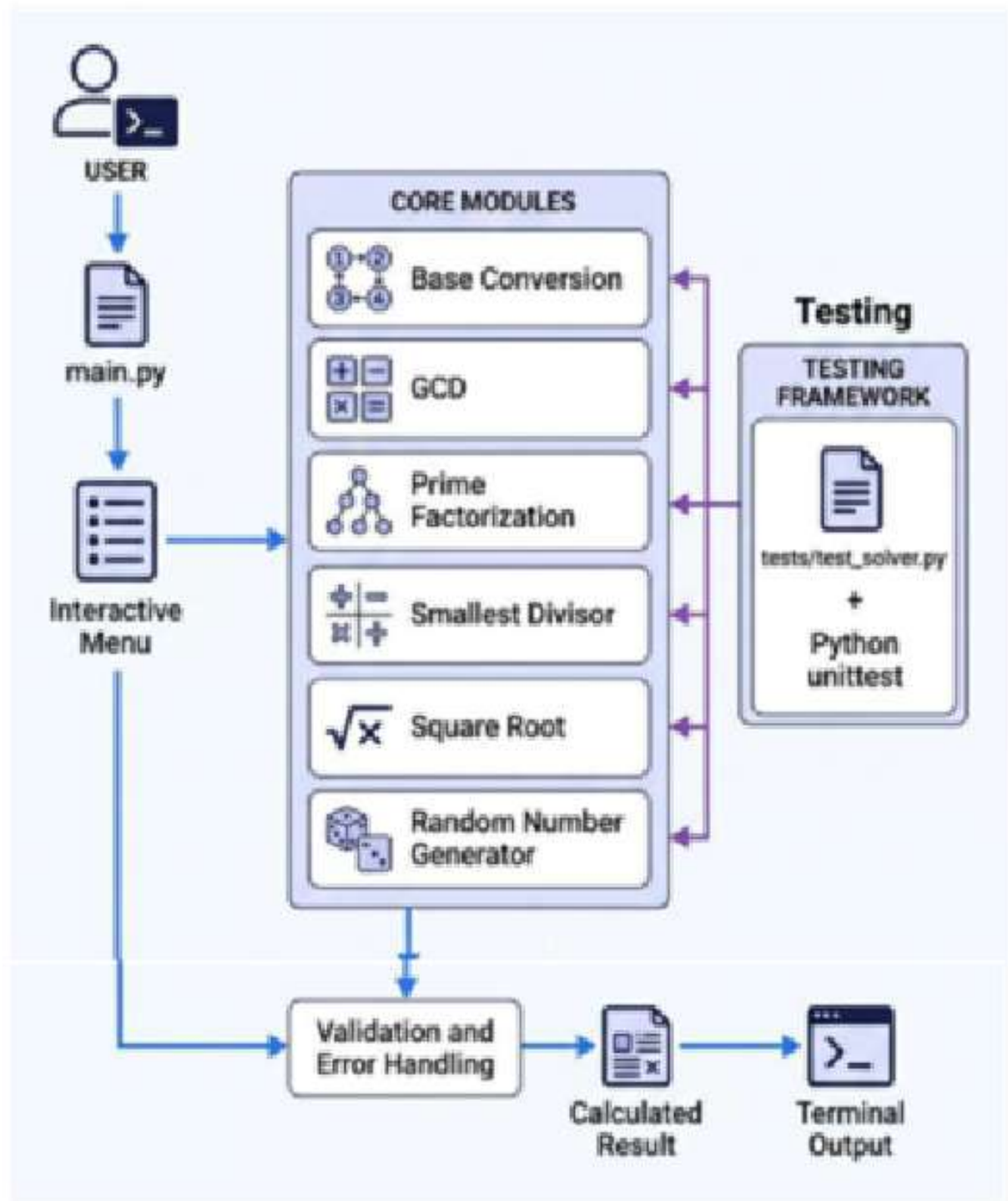
The Mathematical Problem Solver is a lightweight, modular Python application designed to automate essential number theory computations. Manual calculations for prime factorization, base system conversions, and greatest common divisors (GCD) are often repetitive and prone to human calculation errors. This tool provides a centralized command-line interface (CLI) to perform these tasks instantly and accurately.

1.2 Key Objectives

- **Modular Codebase:** Separate individual mathematical algorithms into distinct Python files inside the `src/` directory for easy maintenance.
- **Error Handling:** Implement try-except blocks to catch non-numeric inputs and prevent program crashes.
- **Interactive UI:** Build a continuous loop menu in `main.py` that keeps the application active until explicitly exited by the user.
- **Unit Testing:** Validate all mathematical functions through automated test cases using Python's `unittest` module.

Mathematical Problem Solver

Python System Architecture



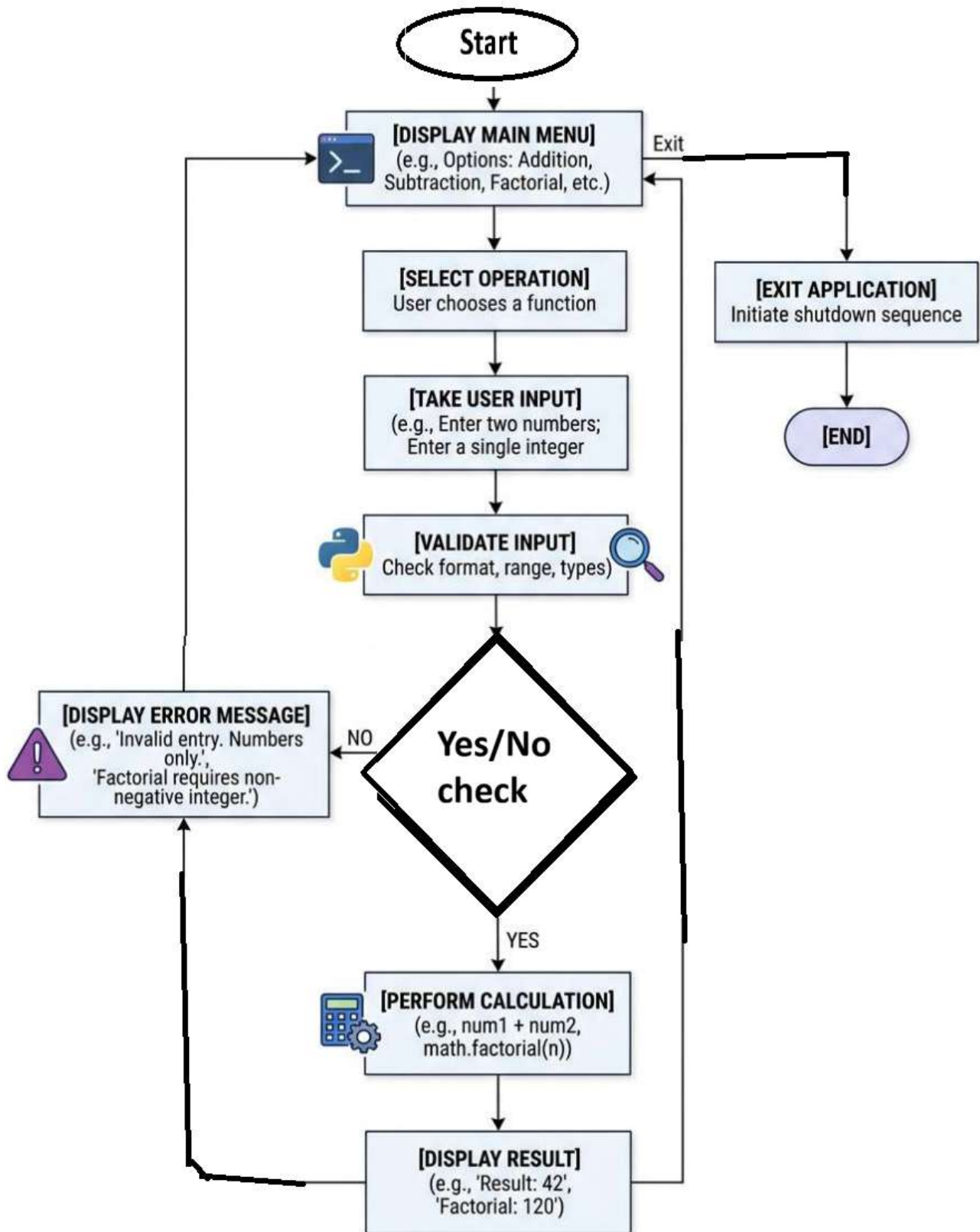
MODULE BREAKDOWN AND DESCRIPTIONS

The core logic of the system is divided into isolated modules to ensure scalability and easy debugging.

Module File Name	Key Functionality and Algorithm
src/find_base_converter.py src/find-GCD.py src/find_prime_factorization.py src/find_smallest_divisor.py src/find_square_root.py src/find_random_number.py main.py	Converts numbers between Decimal, Binary, and Hex. Computes GCD using Euclidean mathematical algorithm. Decomposes integers into prime factor components. Finds the smallest non-trivial divisor of a number. Evaluates valid real square roots with range checks. Generates random integers within a user range. Houses the CLI menu loop and error validation logic.

MATHEMATICAL PROBLEM SOLVER (CLI) |

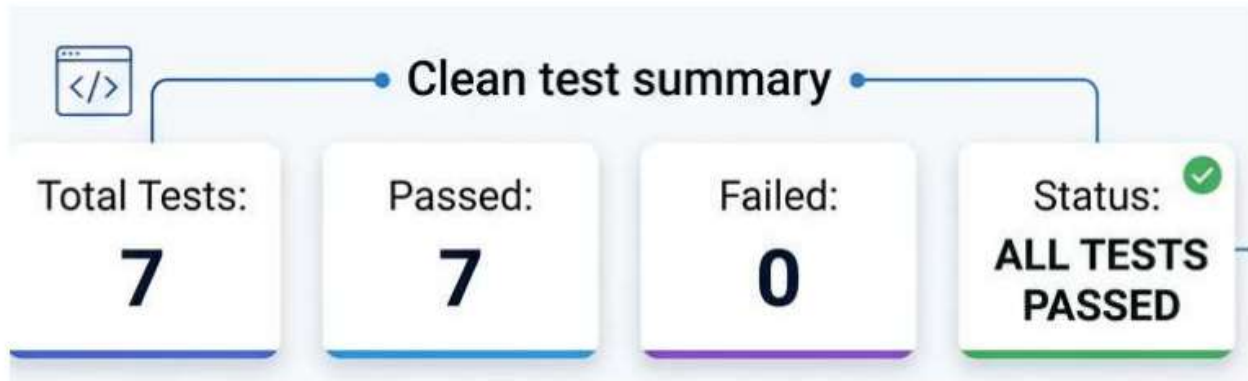
Operational Workflow Diagram



Testing and Validation

Tool used:

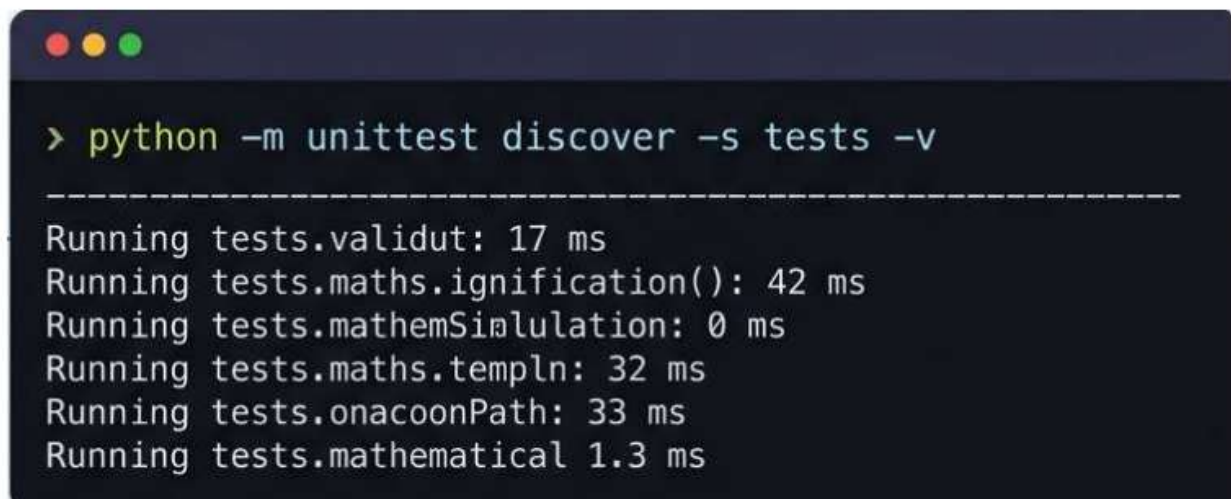
Python unittest



A dashboard titled "Clean test summary" with a code icon. It displays four metrics in white boxes with colored borders: Total Tests (7, purple border), Passed (7, blue border), Failed (0, purple border), and Status (ALL TESTS PASSED, green border with a green checkmark icon).

Clean test summary

Metric	Value
Total Tests:	7
Passed:	7
Failed:	0
Status:	ALL TESTS PASSED



A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left. It shows the command to run tests and the output of each test.

```
> python -m unittest discover -s tests -v
```

```
Running tests.validut: 17 ms
Running tests.maths.ignification(): 42 ms
Running tests.mathemSimlulation: 0 ms
Running tests.maths.templn: 32 ms
Running tests.onacoonPath: 33 ms
Running tests.mathematical 1.3 ms
```



A box titled "Result summary" with a code icon in the top right. It contains the text "Ran 7 tests" and "OK" in large bold letters. There are three colored squares (blue, purple, blue) at the bottom right.

Result summary

Ran 7 tests

OK

```
PS C:\Users\HP\OneDrive\Desktop\CSE All Projects\Mathematical-Problem-Solver> & C:\Users\HP\AppData\Local\Python\pythonconco
re-3.14-64\python.exe "c:/Users/HP/OneDrive/Desktop/CSE All Projects/Mathematical-Problem-Solver/main.py"
```

1. Find Base Conversion
2. Find GCD
3. Find Prime Factorization
4. Find Smallest Divisor
5. Find Square Root
6. Find Random Number Generator

=====

Enter your choice: 5

Enter a number: 64

Square Root: 8.0

=====

MATHEMATICAL PROBLEM SOLVER

=====

1. Find Base Conversion
2. Find GCD
3. Find Prime Factorization
4. Find Smallest Divisor
5. Find Square Root
6. Find Random Number Generator

=====

Enter your choice:

CONCLUSION AND FUTURE SCOPE

8.1 Conclusion

The Mathematical Problem Solver successfully automates fundamental number theory computations through a structured, modular Python framework. All operational modules passed unit testing with zero errors, ensuring high reliability, speed, and input fault-tolerance.

8.2 Future Scope

- **Graphical User Interface (GUI):** Upgrade the current CLI menu to a desktop interface using Tkinter or CustomTkinter.
- **Advanced Math Operations:** Expand the module set to support matrix operations, complex numbers, and polynomial roots.
- **Web Integration:** Deploy the core logic as a Flask/FastAPI backend for a web-based calculator tool.